Progressive Retrieval Attention for Pretrained Transformers: Sparse Native-K/V Memory for Qwen and Llama

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 12, 2026

Abstract

Dense attention makes every retained token part of every subsequent operation. Progressive Retrieval Attention (PRA) instead keeps detailed, layer-native key/value (K/V) state outside the active context, indexes it with compact semantic gists, and materializes only selected native K/V under a fixed token budget. This paper asks whether that mechanism can be added beneath pretrained Hugging Face decoders without changing their ordinary behavior. Thin Qwen and Llama-family adapters pass exact disabled-path tests for logits, hidden states, and greedy generation; cache shapes match on Qwen, and every cache K/V tensor matches on SmolLM2. The adapters preserve each model's rotary, grouped-query, mask, cache, and output-projection semantics. The central design result is that routing state should not be attention state: contextual hidden states support semantic selection, whereas selected tokens must return as the model's own positional K/V.

On matched HotpotQA and QASPER evidence-ranking tasks, a 262,144-parameter router trained over a frozen backbone raises held-out recall@3 from 0.469 to 0.631 ± 0.069 across five seeds. More informative fraction-based evaluation reaches any-evidence recall 0.831 at 10% and 0.956 at 20% of parent chunks, although cross-domain recall@3 remains 0.213. Oracle causal tests show that correct native memory changes the frozen model: on HotpotQA, final-four-layer replay improves gold-token log-probability by 1.004 nats/token, compared with 4.635 for direct evidence text. Generated F1 does not improve, and QASPER's direct-text control fails, so these are mechanistic rather than solved-QA results. The implementation packages exact routing, fraction-based selection, bounded materialization, CPU-resident detail K/V, and route-once multi-layer replay behind a small public API. Five-seed QASPER runs produce evidence-identity recall@20% of 0.454 ± 0.007 on Qwen3-0.6B and 0.447 ± 0.051 on SmolLM2-135M. PRA-HF therefore establishes exact retrofit, sparse discovery, and causal use of native K/V as separate measurable capabilities; robust decoded-answer gains and broad cross-domain routing remain open.

1 Introduction

The cost of long context is not only storing old tokens. As long as every token remains active, each new query must attend over all of them and every decoder layer must carry their K/V state on the accelerator. A system may have abundant CPU memory or external storage and still be unable to make a long document an ordinary dense prompt. Prompt truncation avoids the cost by discarding history. Text retrieval reduces the prompt before inference, but then re-encodes retrieved text and exposes selection only at document or passage boundaries. PRA explores a third boundary: retain the model's already computed internal detail, keep most of it off the active path, and bring back only the native K/V for chunks selected for the present query.

That idea is easy to state and surprisingly easy to implement incorrectly. A pretrained decoder has learned a particular projection layout, rotary coordinate system, grouped-query expansion, mask convention, cache protocol, and output projection. A replacement attention block can appear plausible while silently changing one of those contracts. Retrieval creates a second problem: the representation best suited to deciding whether a chunk is relevant need not be the representation that the model expects to attend to after selection. This paper treats exact transport, sparse discovery, and useful consumption as three different questions.

Table 1: Headline findings. "Exact" means equality in the disabled path, not approximate agreement. Recall definitions and cohort sizes are given with the corresponding experiments.

Question	Measurement	Result
Can PRA be an exact retrofit?	Disabled logits, hidden states, greedy output; cache contract	Exact outputs on both; cache shapes on Qwen and tensors on SmolLM2
Can compact gists find evidence?	Five-seed router over frozen Qwen	Any-evidence recall 0.831/0.956 at 10/20% selected parents
How small is the learned index?	Asymmetric 128-D Qwen router	262,144 parameters (0.044%); 0.392% of per-layer detail K/V
Does routing scale at fixed sparsity?	12.5% selection over 64/128/256 units	Coverage remains 0.756–0.772 (Hotpot) and 0.794–0.803 (QASPER)
Does selected native K/V affect the model?	HotpotQA oracle, final four layers	+1.004 nats/token; generated F1 unchanged
Does the interface cross families?	QASPER evidence-identity recall@20%	Qwen 0.454 ± 0.007; SmolLM2 0.447 ± 0.051
Is there a reusable implementation?	Public API, CLI, router artifacts, and metrics	Yes; exact-search research implementation
Is long-context QA solved?	Product-path generation	No; routed F1 remains 0.033/0.028 in a four-example probe

Our contribution is a working boundary, not a new pretrained model. A shared execution core owns reference publication, compact indexing, exact routing, whole-chunk budgeting, transfer, and metrics. Thin family adapters own only the host model's native attention semantics. Qwen3-0.6B is the main mechanistic checkpoint because it exercises modern RoPE and GQA at a size that allows exact testing on constrained hardware. SmolLM2-135M tests whether the same boundary transfers to a Llama-family implementation. The release adds a stable router artifact, a Python API and CLI, and machine-readable accounting for requested versus physically materialized memory.

The experiments are deliberately layered. WikiText-2 in the controlled predecessor establishes ordinary language-model integrity. QASPER supplies the primary one-shot evidence retrieval task, where a question typically points into a long scientific document. HotpotQA is the harder multi-hop stress test: finding any supporting passage is useful, but finding every required piece is stricter. We report *any-evidence recall*, *evidence-identity recall* (fraction of annotated identities recovered), and *all-evidence recall* separately. A high value for one must not be read as a high value for the others.

2 PRA-HF in 60 Seconds

Figure 1 is the entire mechanism. A long source is encoded in blocks that fit the host model. At each PRA-enabled layer, publication keeps two products. The first is a tiny index: one or more semantic gists per routing chunk. The second is the detail payload: the exact native post-RoPE keys and native values produced by that layer. Detail may remain on CPU. At query time, a

compact query state scores all gists, a policy chooses parent chunks, and a hard budget may trim that choice. Only then are the selected native K/V tensors moved to the accelerator and prepended to the direct K/V sequence under the model's own attention softmax.

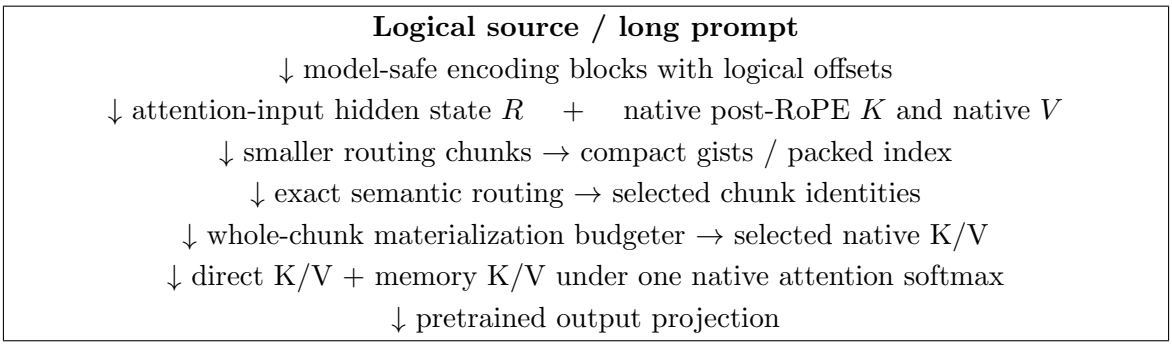

Figure 1: Canonical PRA-HF path. Routing uses compact semantic state, whereas attention receives the host model's selected native positional K/V payload. Model-family projection, RoPE, GQA, mask, cache, and output semantics remain in the thin adapter.

This division answers the most important implementation question. PRA does *not* attend to the gist. The gist is an address label, analogous to a compact catalog entry. Once an address is chosen, attention reads the original native K/V payload. Consequently, routing can use a semantic hidden state without asking a pretrained attention head to consume an unfamiliar representation. Likewise, positional correction is a routing concern only when positional keys are used as gists; the selected attention payload always retains its original native rotary encoding.

The budget distinguishes logical availability from physical activation. A reference may contain thousands of cached tokens, while one operation materializes only B selected tokens plus the direct sequence and a small safety reserve. The mechanism therefore bounds active attention even when the logical source is longer. It does not make storage free: full layer-native detail K/V still occupies CPU memory unless a separate compression or tiering method is added.

Table 2: Three independent success layers. Later evidence cannot repair a failure in an earlier layer, and success in an earlier layer does not imply success in a later one.

Layer	Question	Present status	Evidence
Transport	Is native behavior preserved and bounded memory delivered correctly?	Yes	Exact disabled parity; native GQA K/V; bounded CUDA probes
Discovery	Can a compact index find useful chunks sparsely?	Strong but incomplete	0.831 any-evidence recall at 10%; weak cross-domain transfer
Causal use	Does retrieved state help the frozen decoder?	Yes in logits; decoded gains open	HotpotQA oracle +1.004 nats/token at four late layers; F1 unchanged

3 Minimal PRA Recap

For layer ℓ, PRA stores reference chunks as native $K_c^{(\ell)}, V_c^{(\ell)} \in \mathbb{R}^{H_{kv} \times T_c \times d_h}$ and one or more compact gists $g_c^{(\ell)}$. A query representation $r^{(\ell)}$ scores the gists, and a global budgeter selects whole chunks.

Materialized memory precedes the direct K/V sequence under one softmax:

$$\mathrm{Attn}(Q, [K_M; K_D], [V_M; V_D]) = \mathrm{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M\right)[V_M; V_D]. \tag{1}$$

This is not a separately normalized cross-attention residual. The native output projection is applied once to the combined result.

Paper 1 established controlled native-K/V routing and materialization. Paper 1.5 separated logical source position from contextual fragmentation. Continuous sources use $p_i = p_{\text{logical start}} + i$, and ordinary post-RoPE keys remain the canonical cache state. The present adapter preserves that contract by capturing each selected HF layer's actual post-RoPE keys and by assigning source-relative positions across bounded encoding blocks.

4 Exact Retrofitting: Shared Core, Thin Adapters

4.1 Two representations, two jobs

PRA represents a cached chunk as

$$\mathcal{M}_c = (R_c, K_c, V_c), \tag{2}$$

where R_c is a compact routing representation and (K_c, V_c) is the payload used after selection. The objects need not inhabit the same feature space. The canonical HF configuration in this paper sets R_c to one or more gists of the hidden state entering the selected attention block, while K_c is that block's native post-RoPE key tensor and V_c its native value tensor. Token-level hidden states are transient during publication; only compact gists and detail K/V persist.

This split follows the specialization chain

$$h_{\text{in}} \longrightarrow W_K h_{\text{in}} \longrightarrow \mathrm{RoPE}(W_K h_{\text{in}}). \tag{3}$$

The first representation retains broad contextual information, the second is an attention-address representation, and the third conditions that address on token position. Native token attention needs the third. Chunk retrieval may prefer the first. The routing query therefore uses the final attention-input hidden state and is compared only with hidden-state gists. It is never compared directly with Q_{post}. This is an empirical default for the tested Qwen checkpoint, not a claim that hidden states are universally optimal routers.

4.2 Shared execution core

The controlled model's former monolithic attention forward pass was divided into reusable operations. `prepare_pra_query` selects the newest valid query and maps GQA query groups into the native K/V routing width. `route_memory` calls the exact packed-index router. `budget_selected_memory` enforces

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{native,max}}. \tag{4}$$

`materialize_selected_kv` removes duplicate overlap, preserves row isolation, and moves only retained detail K/V. `combine_local_and_memory_kv` pads variable rows and prepends an additive all-visible memory mask to the family's existing local causal mask. The controlled model and HF adapters now call this same core.

4.3 Thin HF contract

The abstract adapter exposes six family operations: project Q/K/V, apply native position encoding, normalize tensor layout, combine the native mask, invoke PRA attention, and project the output. Disabled memory takes an even narrower path: call the original attention object unchanged. This design preserves pretrained parameters and avoids checkpoint conversion.

The Qwen adapter has 272 physical lines (248 nonblank, non-comment lines). It introduces no learned parameter. Its family-specific branches distinguish Qwen2-style projections from Qwen3's Q/K normalization; routing, budgeting, materialization, residency, and metrics remain outside the file. This is a portability measurement, not a code-minimization target.

4.4 Qwen native semantics

4.4.1 Projection and RoPE

The adapter calls the wrapped module's `q_proj`, `k_proj`, and `v_proj`. Qwen3's existing `q_norm` and `k_norm` are applied before the installed Transformers `apply_rotary_pos_emb` function. Reference capture saves post-RoPE K and position-independent V. No independent RoPE implementation is introduced.

4.4.2 Grouped-query attention

The tested checkpoint has $H_q = 16$, $H_{kv} = 8$, and $d_h = 128$. Cache objects retain $[B, 8, T, 128]$ K/V. Key-space routing baselines average each pair of query heads sharing one K/V head, producing a vector in the same 8×128 space as a flattened key gist. The canonical hidden-state router instead uses one d_{model} vector and does not alter payload head layout. K/V expansion occurs inside Qwen's eager attention function, never in persistent PRA storage. A synthetic replay test compares this path with explicit K/V head repetition and matches within 10^{-6}.

4.4.3 HF cache and masks

Prefill and single-token decode both use the wrapped module's HF `Cache.update` call. PRA memory is prepended after that update, so direct generation history appears once. A decode test with four prompt tokens and two generated tokens observes five direct cache positions at the last invocation, plus four separate memory positions. The original causal mask remains on local/cache positions; selected historical memory receives a visible prefix mask. Memory- disabled generation delegates entirely to the original module.

5 Evaluation Design

Table 3 records the first checkpoint exactly. Tests use synthetic Qwen3 configs offline. The pretrained run uses Python 3.10.11, PyTorch 2.12.1+CUDA 12.6, Transformers 4.55.4, eager attention, FP16, and an NVIDIA GeForce GTX 950M. The frozen model is not fine-tuned.

Every run writes JSON and scalar CSV with the Git SHA, model revision, software versions, device, dtype, layer selection, limits, routing settings, timings, and metrics. The exact executed commit is retained in each artifact rather than inferred from the manuscript build.

Table 3: Pinned Qwen checkpoint and PRA placement.

Checkpoint	`Qwen/Qwen3-0.6B`
Revision	`c1899de289a04d12100db370d81485cdf75e47ca`
Parameters / layers	596,049,920 / 28
Query heads / K/V heads / head width	16 / 8 / 128
RoPE base / advertised positions	10^6 / 40,960
Initial PRA placement	layer 27 only
Depth-study instrumentation / selected band	all 28 layers / layers 20–27
Initial gist / routing policy	mean, one gist/chunk, exact top-k
Native limit / direct / selected-memory cap	256 / 128 / 64 tokens
K/V residency	CPU full cache, selected transfer

6 Exactness and Bounded Execution

6.1 Disabled-path parity

PRA-disabled parity is the hard gate. The baseline is evaluated first; the same in-memory model is then wrapped, leaving every parameter object in place. Table 4 reports exact results for the five-token prompt "The capital of Portugal is" and four-token greedy decode.

Table 4: Original HF model versus memory-disabled PRA wrapper.

Metric	Result
Maximum / mean absolute logit difference	0 / 0
Maximum hidden-state difference	0
Greedy token agreement	100%
Greedy sequences exactly equal	yes
Generation-cache shapes equal	all 28 layers
Finite adapted logits	yes

Single-run prefill times were 0.447 s before wrapping and 0.136 s after wrapping. This order is confounded by first-call warm-up and is not a speed comparison. Exact outputs, rather than latency, are the parity evidence.

6.2 Native-K/V reference transport

An explicit 22-token reference about Lisbon is encoded through the frozen model. The selected layer captures $[1, 8, 22, 128]$ K/V, stores it on CPU, creates a mean gist, and transfers all 22 tokens because the source fits one routing chunk. The physical transfer is 90,112 bytes, exactly $2 \times 22 \times 8 \times 128 \times 2$ bytes for FP16 K and V. The run reports 1.6 ms routing, 0.40 ms materialization orchestration, 0.15 ms selected transfer, and 0.11 ms combined attention. These synchronized phase values are instrumentation checks on one machine, not throughput claims. Cold reference construction took 0.144 s and the warm query took 0.191 s.

The generated continuation contains "Lisbon", but the unmodified model also knows this fact. Consequently, this observation establishes functional selected-K/V transport and generation, not a causal retrieval benefit. The exact native-replay claim is instead restricted to the synthetic tensor test, where GQA memory plus local K/V equals explicit dense composition.

6.3 Implicit head and bounded long prompt

A repeated prompt tokenizes to 408 logical tokens. The direct cap retains the final 128 tokens; the first 280 become `#_head`. That head is encoded in blocks of at most 64 tokens, with source-relative positions. The direct tail receives positions 280 through 407. Every encoding operation and the final selected-memory operation stays under the 256-token configured native limit; the largest observed source-encoding call is 64 tokens and violations are zero. The final finite-logit query took 0.258 s. This proves bounded execution and offset propagation for the frozen Qwen adapter. It does not show that the router selected the information needed by an arbitrary continuation.

6.4 Why transport is not retrieval

We next run one fixed HotpotQA example [6] and one QASPER example [2]. Four conditions use the same frozen checkpoint: question-only truncation, dense text left-truncated to 256 tokens, oracle evidence text-RAG, and PRA with the question direct and the complete source in native-K/V memory. Oracle text-RAG is an upper control because it receives annotated evidence. This two-example matrix diagnoses plumbing; it cannot estimate dataset accuracy.

Table 5: Unrestricted-QA smoke outcomes. Standard normalized token F1 is shown.

Dataset	Condition	F1	Annotated evidence selected
HotpotQA	truncation; dense; oracle text-RAG; PRA	0; 0; 0; 0	PRA: 0%
QASPER	truncation; dense; oracle text-RAG; PRA	0; 0.182; 0; 0.182	PRA: 0%

HotpotQA's expected answer is "yes"; every condition begins with "No". QASPER's expected answer is "no"; dense and PRA generations begin with "No", while truncation begins with "Yes". However, PRA routes three tail chunks, none overlapping the annotated evidence, so the QASPER output cannot be credited to retrieved evidence. Qwen also fails the oracle text-RAG control on both examples. The proper conclusion is negative: this frozen top-layer, one-gist-per-chunk configuration has not demonstrated content-causal retrieval.

The selections expose a concrete failure mode. For HotpotQA, evidence lies at token spans 107–137 and 543–588, while routed chunks lie near 1216–1318. For QASPER, evidence lies at 0–317, while selected chunks lie near 4032–4192. Post-RoPE mean-key gists and an unadapted top-layer query systematically prefer irrelevant late-source chunks in these two probes.

7 Semantic Routing, Native Attention

The smoke failure motivates a narrower test: is the post-RoPE representation appropriate for semantic routing? For token j at position p_j, the baseline pools

$$g_{\text{post}} = \frac{1}{m} \sum_{j=1}^{m} R(p_j) K_{\text{pre},j}, \tag{5}$$

which mixes content with different rotary phases. The position-neutral alternative pools $g_{\text{pre}} = m^{-1} \sum_j K_{\text{pre},j}$. A third representation pools the attention-input hidden states. These are hypotheses about ranking only; selected detail memory remains Qwen's post-RoPE K and native V in every condition.

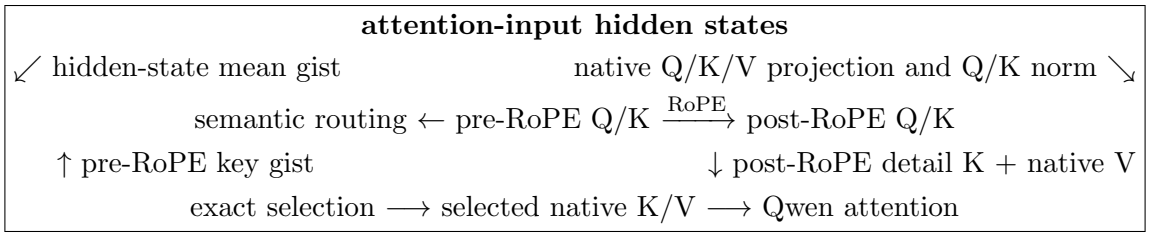

Figure 2: Routing and transport are separate. Pre-RoPE or hidden-state features rank chunks; the attention path always materializes post-RoPE native K/V.

The code makes this independence executable. Routing returns stable *selected chunk identities*; a separate core entry point applies the common budget and materializes their payload. A fixed-selection test supplies the same identity set with ordinary-router and oracle metadata. It obtains bit-identical post-RoPE K, identical V, and exactly identical native attention output. Router choice can change the set, but it cannot silently change what a fixed set means to attention.

7.1 What should represent a chunk?

We first compare three parameter-free summaries on 16 matched examples, eight from HotpotQA and eight from QASPER. All use 32-token parents and the same exact top-k search. Post-RoPE key means mix tokens carrying different rotary phases. Pre-RoPE key means remove position from the index. Attention-input hidden-state means retain broader contextual features. In all three conditions, selection materializes the same post-RoPE native K and native V for a fixed parent identity.

Table 6: Parameter-free routing representations on 16 matched examples. Score-position r measures correlation with normalized source position.

Routing representation	Recall@3	Recall@8	Recall@16	MRR	Position r
Post-RoPE key mean	0.125	0.250	0.313	0.131	0.652
Pre-RoPE key mean	0.313	**0.563**	**0.750**	**0.301**	0.009
Hidden-state mean	**0.438**	0.500	**0.750**	0.276	-0.021

The post-RoPE mean is not merely noisy; it strongly favors late chunks. Removing rotary phase nearly eliminates that bias, and hidden states give the best low-budget result. Centered RoPE offers a useful positive control: rotate a pooled pre-RoPE gist once at the exact center of its span. This raises recall@3 from 0.125 to 0.188, and eight centered subgists reproduce native token-QK ordering with correlation 0.874. Yet their recall@3 is only 0.250. Centering repairs the coordinate frame, but positional compatibility is not semantic relevance. We therefore use hidden-state gists for routing and preserve post-RoPE keys only for attention.

8 Sparse Discovery with a Tiny Learned Router

Parameter-free cosine similarity asks the pretrained hidden space to serve a new retrieval objective without supervision. We instead freeze all 596,049,920 Qwen parameters and learn only two projections,

$$s(q, c) = \mathrm{norm}(W_q h_q)^\top \mathrm{norm}(W_c g_c), \tag{6}$$

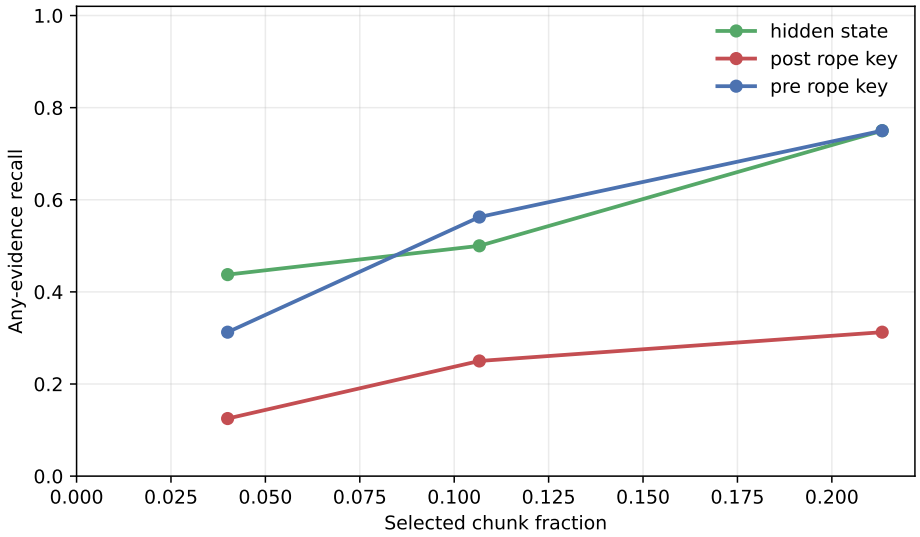

Figure 3: Parameter-free any-evidence recall versus selected-parent fraction. Position-neutral representations dominate the post-RoPE mean. These 16-example curves are diagnostic, not a cross-method statistical comparison with the larger learned-router cohort.

where h_q is the last query state and g_c is one mean hidden-state gist. The selected asymmetric 1024-to-128 linear router contains 262,144 parameters, 0.044% of Qwen. Training uses 48 examples, validation uses 16, and the held-out test uses 32, with no QA-identity overlap. Validation selects an exhaustive pairwise margin objective; five independent seeds measure the selected configuration.

Table 7: Held-out sparse ranking. The interval is the 95% half-width over five router seeds.

Router	Recall@3	Recall@8	MRR	Hotpot R@3	QASPER R@3
Frozen cosine, last state	0.469	0.719	0.413	0.813	0.125
Learned asymmetric-128	$\mathbf{0.631 \pm 0.069}$	0.806	0.498	0.588	0.675
Shuffled-label control	0.125	–	–	–	–

The adapter improves paired recall@3 by 0.163 ± 0.069 and removes measurable position bias ($r = -0.026$). The shuffled-label collapse shows that the gain depends on evidence supervision. It is not yet a general retriever: Hotpot-to-QASPER and QASPER-to-Hotpot transfer both reach only 0.213 recall@3, and the combined 0.70 promotion target is missed. Wider projections, sampled negatives, and one hard-negative refresh do not improve the validation-selected result.

8.1 The useful operating point is a fraction, not a fixed rank

Recall@3 is intentionally severe, but it changes meaning as source length changes. For each example with N_i parents, we also select $k_i(f) = \lceil fN_i \rceil$. Historical artifacts support *any-evidence recall*: whether at least one annotated parent is recovered. Complete rankings support the stricter *evidence-identity recall*: the fraction of all mapped evidence parents recovered. We never substitute one definition for the other.

Table 8: Any-evidence recall versus requested parent fraction. The learned row averages five seeds and 160 seed–example evaluations. Other rows are descriptive historical cohorts.

Mechanism	R@5%	R@10%	R@20%	R@30%	AUC_{0--30}
Post-RoPE key mean	0.125	0.125	0.250	0.375	0.190
Pre-RoPE key mean	0.375	0.563	0.938	1.000	0.700
Hidden-state mean	0.438	0.625	0.625	0.813	0.574
Last-state query	0.531	0.719	0.875	0.906	0.724
Learned margin router	**0.675**	**0.831**	**0.956**	**1.000**	**0.835**

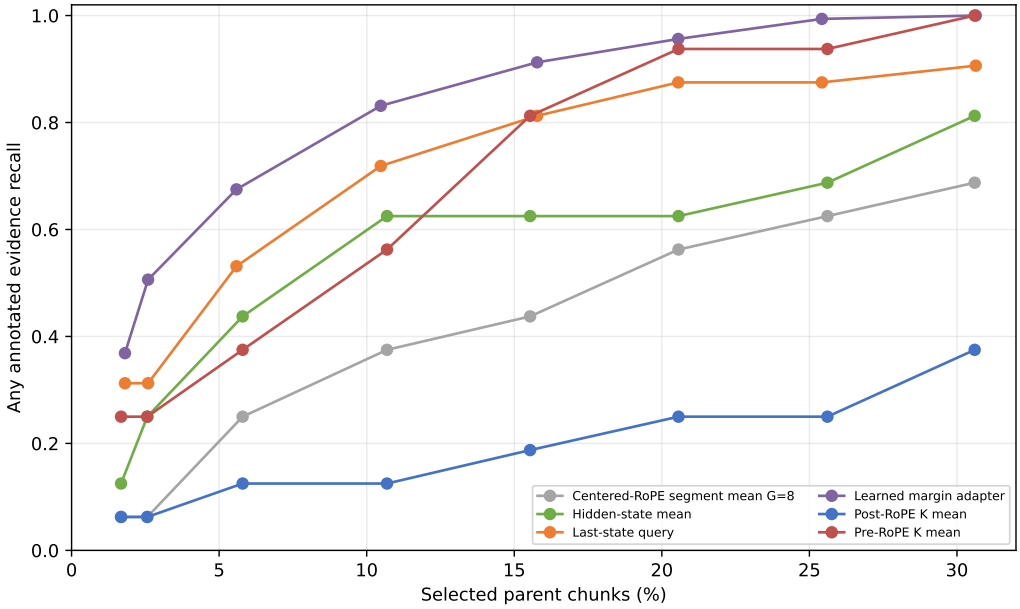

Figure 4: Any-evidence recall over the primary 0–30% region. The learned router has a useful 10–20% operating range that fixed Recall@3 conceals. Cohorts differ for historical baselines, so only the five-seed learned result supports uncertainty across seeds.

At requested 10% and 20%, the selected seed recovers 0.356 and 0.442 of *all* annotated evidence identities, while the corresponding any-evidence values are 0.844 and 0.906. Thus a system can often find one useful passage while still missing other evidence required for a multi-hop answer. This distinction explains why HotpotQA needs a larger realized fraction for 90% any-evidence recall (0.207) than QASPER (0.104).

Scaling must also hold the denominator fixed. In the standalone predecessor, selecting a fixed eight references makes the selected fraction shrink from about 12.5% to 3.1% as memory grows from 64 to 256 units; apparent coverage consequently falls. Selecting approximately 12.5% instead keeps HotpotQA-derived target coverage at 0.756/0.772/0.762 and QASPER-derived coverage at 0.794/0.803/0.800 for 64/128/256 units. Ranking remains stable at a fixed fraction, while physical materialization grows with that fraction. Sparse quality and active-K/V cost must therefore be reported together.

9 Cross-Family Validation

The same artifact and execution interfaces are exercised on Qwen3-0.6B and the public SmolLM2-135M Llama-family checkpoint. Both routers use frozen last-layer attention-input states, one mean per 32-token parent, 128-dimensional asymmetric projections, 512 optimization steps, QASPER training, and seeds 11, 23, 37, 53, and 71. Table 9 reports evidence-identity recall, not the any-evidence metric above.

Table 9: Five-seed QASPER test evidence-identity recall. Values are mean ± sample standard deviation where shown.

Family	R@5%	R@10%	R@20%	R@30%	$AUC_{0:30}$
Qwen3-0.6B	0.293	0.362	0.454 ± 0.007	0.532	0.396
SmolLM2-135M	0.243	0.334	0.447 ± 0.051	0.532	0.371

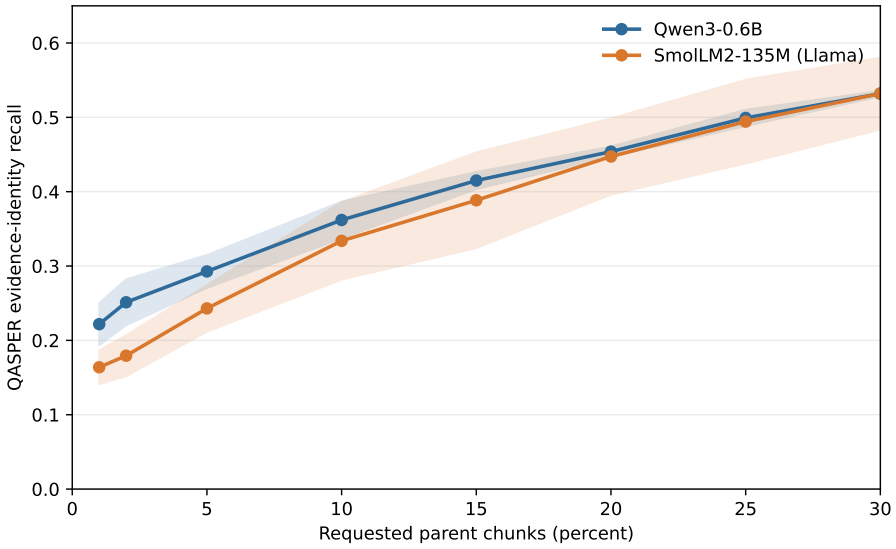

Figure 5: Matched QASPER evidence-identity recall across five router seeds. The interface crosses families, but neither family reaches 0.80 recall before exhaustive selection.

The means at 20–30% are close, but Qwen is more stable and has the stronger sparse AUC. This supports software and tensor-contract portability, not universal transfer of routing quality. The official Meta Llama 3.2-1B checkpoint remains an unmeasured target because its manually gated repository returned HTTP 401 in the available environment. No SmolLM2 number is presented as a Meta Llama result.

10 Causal Use: Does Correct Native Memory Help?

Retrieval recall answers whether a router found annotated evidence. It does not answer whether the frozen decoder can use that evidence after it becomes native K/V. The initial adapter consumed routed memory only at layer 27; the earlier oracle diagnostic activated at most layers 24–27. We therefore intervene at both the selection and consumption boundaries. The no-memory condition

supplies only the question. Oracle conditions force every 32-token parent intersecting an annotated evidence span. The direct-text control places the same evidence in the ordinary prompt. Four deterministic examples from each dataset are used, so this remains a mechanism probe rather than an accuracy estimate.

Forced identities enter `prepare_selected_memory`, exactly where ordinary semantic routing ends. Both routes then share thresholding, whole-parent budgeting, native post-RoPE K/V, CPU-to-GPU transfer, GQA layout, source positions, additive masks, and Qwen's eager attention kernel. A selected parent identity is fixed across layers, but its tensor is not: layer ℓ receives $K_c^{(\ell)}, V_c^{(\ell)}$ captured from that layer's own Qwen projections and RoPE path. The 640-token bound reserves 128 direct tokens and permits up to 512 memory tokens per layer. Every oracle parent survives; no budget or native-limit violation occurs, and each reference-encoding call is at most 128 tokens. "Active K/V" is the sum of materialized physical tokens over consuming layers, not the number of distinct source tokens.

Generation-only F1/EM is too coarse for this small frozen checkpoint. The primary paired outcome is therefore teacher-forced mean gold-token log-probability,

$$\Delta_{c,i} = \frac{1}{|y_i|} \log p(y_i \mid q_i, c) - \frac{1}{|y_i|} \log p(y_i \mid q_i, \varnothing), \tag{7}$$

with first-token probability/rank, generated text, attention mass, and answer-position hidden state deltas retained in per-example traces. Four deterministic examples from each dataset are used. This is a mechanism probe, not an accuracy estimate.

Table 10: Oracle consumption-depth sweep. $\Delta \log p$ is paired mean gold-token log-probability relative to no memory. K/V is aggregate physical materialization across active layers. TTFT and generation are synchronized descriptive seconds, not serving benchmarks.

Dataset	Condition	Layers	$\Delta \log p$	F1	K/V tokens	TTFT	Generation
HotpotQA	none	0	.000	.056	0	.140	.737
HotpotQA	oracle, last 1	1	+.071	.056	112	.143	.733
HotpotQA	oracle, last 4	4	+1.004	.056	449	.154	.813
HotpotQA	oracle, last 8	8	+2.311	.056	898	.169	.877
HotpotQA	oracle, last half	14	+3.641	.056	1,572	.193	1.013
HotpotQA	oracle, all	28	-.766	.000	3,143	.245	1.382
HotpotQA	direct evidence text	0	+4.635	.125	0	.198	.986
QASPER	none	0	.000	.125	0	.135	.781
QASPER	oracle, last 1	1	-.091	.125	240	.137	.785
QASPER	oracle, last 4	4	-.161	.125	960	.149	.844
QASPER	oracle, last 8	8	+.347	.125	1,920	.163	.899
QASPER	oracle, last half	14	+.072	.100	3,360	.188	1.081
QASPER	oracle, all	28	-7.492	.000	6,720	.249	1.488
QASPER	direct evidence text	0	-1.791	.000	0	.416	1.090

HotpotQA supports the integration-depth hypothesis within a late band. The last-one, last-four, last-eight, and last-half interventions move mean log-probability by +0.071, +1.004, +2.311, and +3.641 nats/token. The last eight recover 49.9% of the direct-text effect while using 8/28 layers; the last half recover 78.6% at 1.75 times the last-eight materialization. Generated F1 does not change. The evidence therefore establishes causal likelihood use, not decoded QA success.

QASPER separates perturbation from interpretable utility. It turns positive at the last eight layers (+0.347) but is only +0.072 at the last half, while direct evidence text is negative (−1.791).

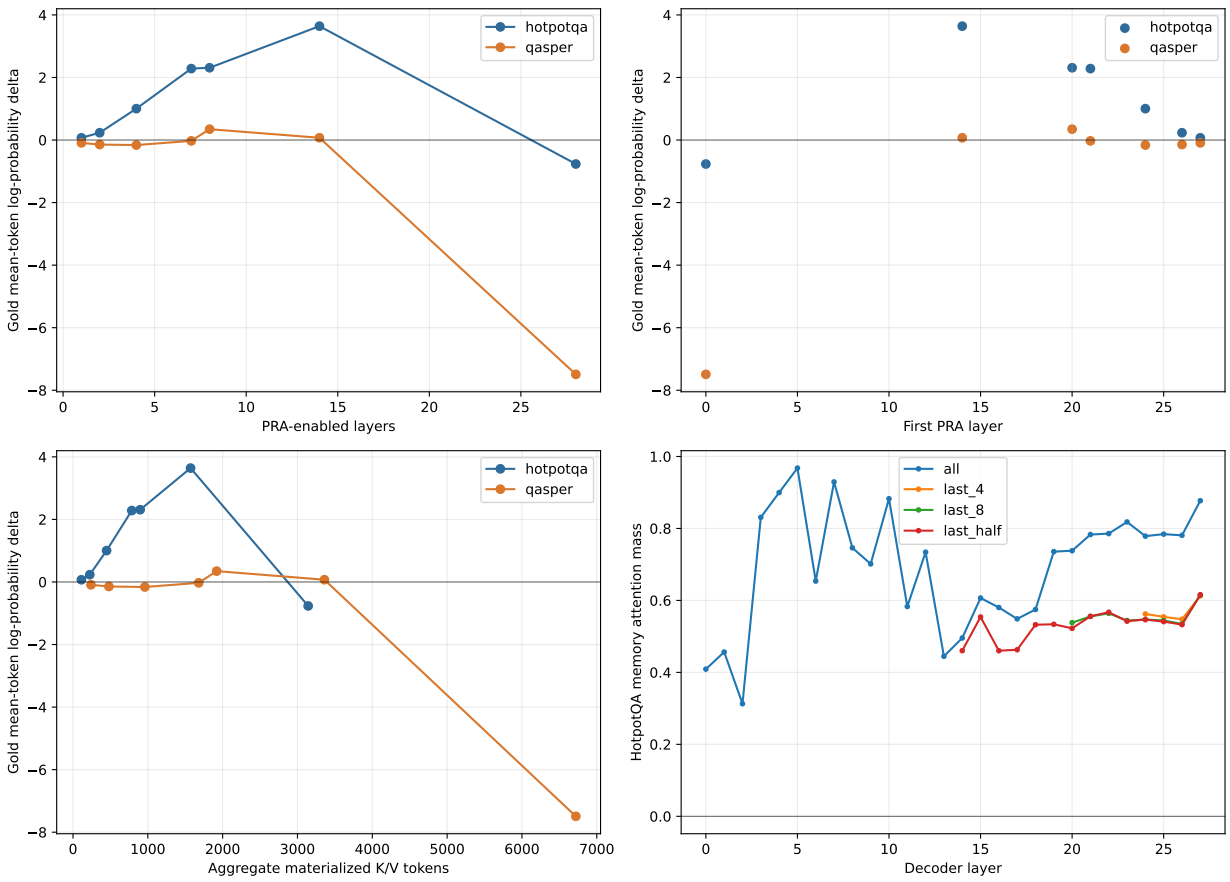

Figure 6: Oracle depth sweep. Repeated exposure through a contiguous late band improves HotpotQA likelihood, but the curve is non-monotone: all-layer replay is harmful on both datasets. The lower panels expose the associated aggregate materialization cost and per-layer attention rather than treating layer count as free.

That control failure prevents using this subset to grade PRA transport. Attention mass and hidden-state perturbations are retained in the artifact, but neither alone is evidence that a perturbation is useful.

10.1 Placement is not interchangeable

The depth result justifies one placement follow-up. Table 11 fixes either four or seven consuming layers and leaves oracle parent identities, positions, prompts, and per-layer payload sizes unchanged.

Table 11: Combined oracle placement result. Equal-count rows have equal aggregate K/V cost.

Placement	Layers	First layer	$\Delta \log p$	F1
Early contiguous	4	0	-10.573	.000
Middle contiguous	4	12	-.556	.090
Evenly spaced	4	0	-10.398	.000
Late contiguous	4	24	+.421	.090
Every fourth	7	0	-6.090	.000
Late contiguous quarter	7	21	+1.128	.090

This rejects both proposed shortcuts. Earlier insertion does not help by leaving more downstream depth, and every-fourth insertion does not approximate dense late consumption. The likely mechanism is a layer-specific contextualization/topology mismatch: independently encoded reference states are least compatible with early residual computation, whereas a contiguous upper band can repeatedly integrate them after the local question has become semantically formed. This interpretation is mechanistic, not yet a general theorem about decoder depth. All-layer replay is especially informative: more attention opportunities can make the answer worse. The operating point is therefore a band, not "as many PRA layers as possible."

10.2 Route once, consume with layer-native payloads

The routed follow-up scores the question once at layer 27, selects three parent IDs, and reuses those identities across either the last 8 or last 14 layers. Every layer resolves the ID to its own post-RoPE K/V; no tensor crosses layer boundaries. Table 12 reports the complete pre-top-k evidence-identity ranking and the aggregate consumption cost.

Table 12: Combined route-once result over eight examples. Recall is the fraction of mapped evidence-parent identities recovered at a parent-selection fraction; f_{80} and f_{90} are the first measured fractions reaching those recall levels.

Router	Consumption	R@5%	R@10%	R@20%	R@30%	f_{80}	f_{90}	$\Delta \log p$	F1	K/V
Learned margin	last 8	.264	.403	.521	.639	.502	1.000	+.227	.090	768
Learned margin	last half	.264	.403	.521	.639	.502	1.000	+.374	.028	1,344
Raw hidden-state cosine	last 8	.092	.224	.414	.573	1.000	1.000	+.150	.090	768
Raw hidden-state cosine	last half	.092	.224	.414	.573	1.000	1.000	+.568	.090	1,344

The learned router gives the stronger sparse ranking and reaches evidence in 5/8 selected top-3 sets. Last-eight consumption preserves the no-memory F1 and moves likelihood by +0.227; last-half consumption moves it by +0.374 but lowers F1 to .028. Raw cosine has weaker sparse recall yet a larger last-half likelihood shift. This is not a contradiction: a selected non-annotated parent can still change the answer distribution, and a likelihood movement need not cross the greedy

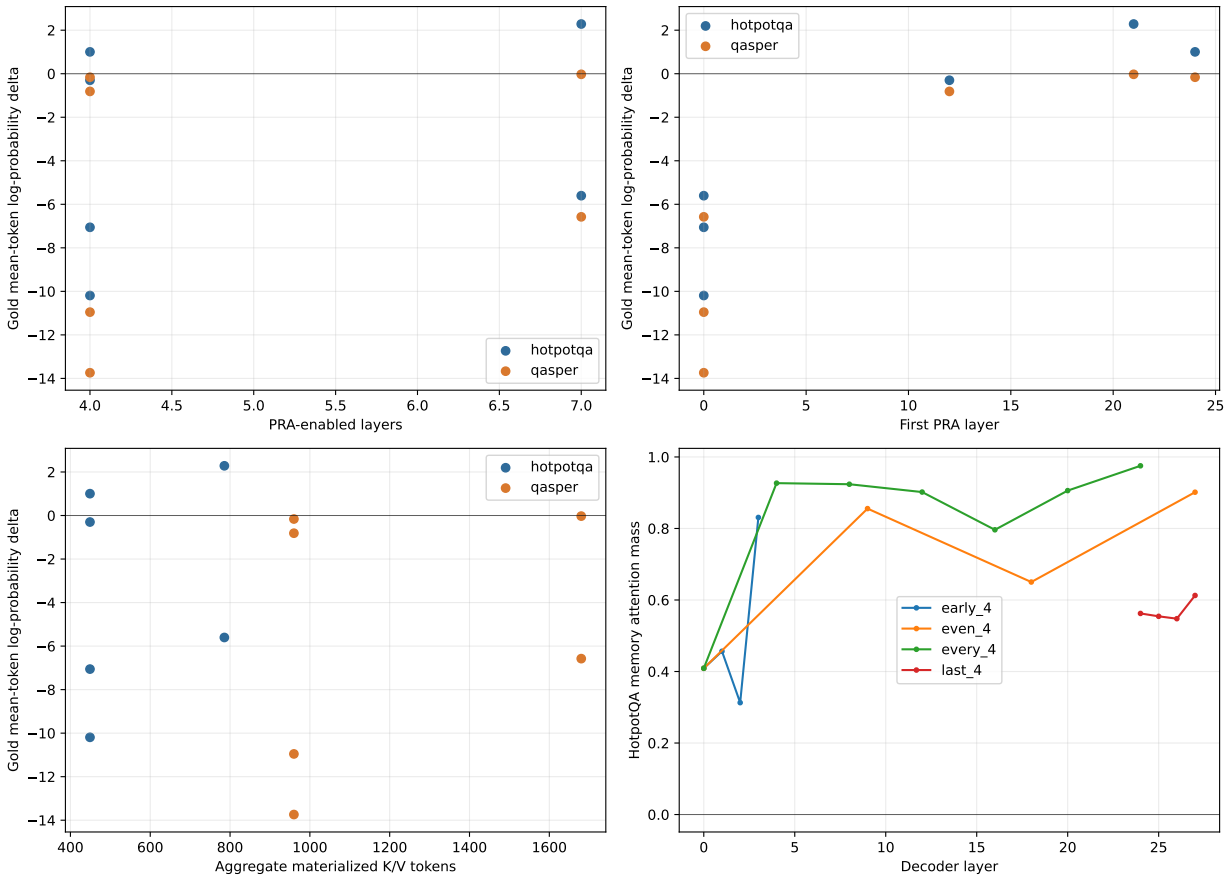

Figure 7: Oracle placement at matched parent identities and matched per-layer payload. Earlier and sparse placements are not approximations to a late contiguous band; they reverse the likelihood effect despite comparable attention and exactly equal K/V cost at equal counts.

decoding boundary. The learned last-eight condition is the conservative quality/cost operating point. It materializes 768 K/V tokens across layers, increases TTFT from .141 to .167 seconds and generation from .759 to .907 seconds, and adds one .149-second query encoding plus .021-second exact route. Its layer-27 learned index averages 45,184 bytes; storing projected gists for all 28 instrumented layers averages 1.27 MB, versus 322.7 MB of resident detail K/V. These eager single-example timings characterize this implementation only.

10.3 Canonical end-to-end confirmation

The earlier single-layer `#__head` run established bounded displacement but generated no correct answer. We repeat that probe with the learned router at layer 27 and the selected late-eight consumption band. The 416-token logical prompt displaces 288 tokens, retains a 128-token direct tail with positions 288–415, and indexes nine 32-token parents. The router ranks the evidence parent first and selects it with two neighboring parents. Each layer 20–27 then consumes its own 96-token native K/V payload.

Table 13: Multi-layer bounded-head confirmation. Rank is the gold first-token vocabulary rank; K/V is aggregate physical materialization over the consuming layers.

Condition	Evidence rank	$\Delta \log p$	First-token rank	K/V	F1	Viol.
No memory	–	.000	94,106	0	.000	0
Learned route, last 8	1	+5.469	1,351	768	.000	0

The intervention raises the two-token answer's mean log-probability from -10.924 to -5.455 and its first-token probability from 3.79×10^{-9} to 1.95×10^{-5}. Greedy output still omits "cobalt." The maximum native operation remains 128 tokens and no violation occurs. Thus the bounded head now demonstrates routing, multi-layer layer-native transport, and strong causal likelihood use together. It still does not establish decoded task success.

11 Product and Systems Boundary

The validated path is packaged as `pra-hf`, with a stable Python API, CLI, serializable configuration, fraction-based selection, and Hugging Face-style router artifacts. A caller adds a reference and generates through the same model object:

```
from pra_hf import PRAConfig, PRAForCausalLM

model = PRAForCausalLM.from_pretrained(
    BASE_MODEL, routing_adapter=ROUTER,
    pra_config=PRAConfig(selected_fraction=0.20))
ref = model.add_reference_file("paper.txt")
result = model.generate(question, return_details=True)
model.remove_reference(ref)
```

The API also supports in-memory text, explicit URI/text pairs, chat formatting, `#__head` rollover, cache clearing, and per-request statistics. A router artifact contains projection weights, configuration, and a model card; it pins the base model revision, family, routing layer and representation, parent size, dataset splits, seeds, metrics, PRA version, and source revision without

republishing base-model weights. Qwen's router has 262,144 parameters (0.044% of 596,049,920); SmolLM2's has 147,456 (0.110% of 134,515,008). Both store one 128-dimensional FP32 vector, 512 bytes, per parent.

Memory accounting uses three denominators. *Routing-index bytes* are the compact vectors searched once per request. *Resident detail bytes* are all layer-native K/V retained on CPU. *Materialized bytes* are the selected K/V transferred for each consuming layer. A 20% requested parent fraction need not become 20% active K/V: whole-parent rounding and the hard token cap may reduce it. Conversely, replaying one selected identity through eight layers incurs eight layer-specific payloads even though semantic routing happened once.

Table 14: Four-example CUDA product-path means on a GTX 950M. This is a systems demonstration, not a throughput or QA benchmark. Peak GPU allocation includes the full model.

Family	Evidence	F1	Requested	Materialized	Route ms	Peak GiB
Qwen3-0.6B	0.667	0.033	20.4%	6.61%	6.40	1.126
SmolLM2-135M	0.750	0.028	21.0%	6.25%	32.74	0.261

Qwen averages a 45.9 KiB routing index, 91.3 MiB of resident eight-layer detail K/V, and 4.0 MiB transferred per request across the eight consuming layers. SmolLM2 averages 48.8 KiB, 18.2 MiB, and 0.75 MiB, respectively. Mean wall time for 16 routed output tokens is 2.15 seconds for Qwen and 1.85 seconds for SmolLM2, with no native-limit violations. These values describe an eager Python implementation with exact Torch search. They are not evidence of serving speedup: there is no matched dense long-context baseline, fused kernel, asynchronous transfer, or concurrency study.

The public implementation is nevertheless a research contribution. It makes the tested boundary reusable, preserves native GQA storage rather than permanently expanding K/V heads, reports both requested and actual materialization, and permits exact disabled-path testing for a new family. The thin Llama adapter resolves the installed rotary and eager-attention functions while the shared core retains reference publication, routing, budgeting, and replay. Full code-level guidance appears in Appendix B.

12 Limitations and Next Experiments

The evidence base is still small. The primary representation study uses 16 matched examples, the learned router has 32 held-out test identities, and the oracle depth study uses only four examples per dataset. The product path uses two held-out examples from each dataset. Confidence across five training seeds measures optimization variation on a fixed cohort; it does not replace a larger test set. The HotpotQA likelihood direction therefore needs independent replication.

Retrieval labels are also imperfect proxies for answer sufficiency. Any-evidence recall can be high while a multi-hop model misses another required passage. Evidence-identity recall improves that accounting but still assumes the annotations identify what the checkpoint needs. The HotpotQA direct-text control is positive and makes its oracle result interpretable. QASPER's direct-text control is negative on the causal subset, so QASPER cannot grade native-K/V utility there. Neither result establishes decoded QA improvement.

The router remains domain-bound. Its 0.213 cross-domain recall@3 in both directions is the clearest warning against a general semantic-memory claim. More dimensions, contiguous subchunk means, token-level maxima, centered-RoPE gists, question-span pooling, sampled negatives, and

one hard-negative refresh do not solve that problem. The next routing experiment should increase identity count and domain diversity before adding capacity, and should report any-, all-, and identity recall against both selected-parent and exact K/V fractions.

The systems work establishes bounded execution and a portable API, not speed. Detail K/V remains resident on CPU, exact search is eager, and selected rows are padded. There is no matched dense long-context baseline, fused attention, ANN index, asynchronous transfer, batching, or concurrency study. The official Meta Llama 3.2-1B run is also absent because checkpoint access was denied; the paper reports SmolLM2 only as a measured Llama-family control.

The most discriminating next experiment is a larger checkpoint-solvable cohort at the final-eight operating point, with oracle, routed, direct-text, and no-memory conditions evaluated together. It should measure gold likelihood, calibrated first-token rank, decoded EM/F1, exact memory bytes, TTFT, and per-token latency. If likelihood improves reproducibly while decoding does not, a small memory-use adaptation becomes justified. If oracle native K/V fails where direct text succeeds, the integration representation, not routing capacity, remains the priority.

13 Related Work

PRA differs from text retrieval-augmented generation, which retrieves text and lets the model encode it in the current prompt [4]. It retrieves layer-native K/V after a separate bounded encoding path. This makes PRA complementary to RAG: a text retriever may choose documents before PRA routes chunks inside cached model state. Sparse-attention systems such as Longformer and BigBird constrain token-token connectivity inside a forward pass [1, 7]; PRA instead materializes selected state from an external addressable cache. KV-cache systems emphasize serving capacity and token retention policies [3, 8, 5]. Paper 2 asks whether selection can be inserted beneath an existing pretrained family without changing its disabled behavior.

14 Conclusion

PRA-HF makes long external state addressable without asking a pretrained decoder to keep every token active. The implementation encodes references in model-safe blocks, retains their native layer K/V on CPU, searches a much smaller semantic index, and materializes selected whole chunks under a hard bound. When disabled, the adapters are exact. When enabled, they preserve native RoPE, GQA layout, masks, cache growth, and output projection across Qwen and Llama-family models.

The main architectural lesson is simple: route semantic state and attend native positional state. Hidden-state gists avoid the strong position bias of post-RoPE means, while fixed selected identities always recover the same post-RoPE K and native V. A 0.044%-of-model router then finds some annotated evidence in 83.1% of cases at 10% selected parents and 95.6% at 20%. That is a credible sparse discovery regime, but weak cross-domain transfer prevents a general retrieval claim.

Correct memory can matter to the frozen decoder. HotpotQA oracle replay through the last four layers improves gold-token log-probability by 1.004 nats/token, while all-layer and early-layer replay are harmful. The effect is therefore about compatible placement, not simply adding more memory-attention sites. Generated F1 remains unchanged, and QASPER's failed direct-text control leaves that subset non-diagnostic. The present result is causal use in logits, not solved long-context question answering.

These boundaries are the point of the paper. Exact transport is established. Sparse discovery is strong and improving but domain-limited. Causal use is visible in likelihood, while reliable decoded

gains remain open. The released API, artifacts, metrics, and reimplementation guide make that frontier reproducible without concealing what has not yet been achieved.

A Routing Ablations

This appendix records the secondary interventions that shaped the final design. They are useful negative results: each changes one plausible source of routing error while leaving native detail K/V unchanged.

Table 15: Parameter-free diagnostics with 32-token parents. Cohorts differ as described in the artifact metadata, so rows summarize directions rather than a single paired leaderboard.

Intervention	Sparse result	Cost or control	Interpretation
Hidden segment means, $G = 1/2/4/8$	R@3 0.391/0.313/0.266/0.281	Index overhead 1.57/3.14/6.28/12.56%	Finer untrained means do not improve sparse ranking
Token-level hidden maximum	R@3 0.375 versus mean 0.438	Index overhead 50% versus 1.57%	Resolution helps some Hotpot ranks but hurts QASPER
Centered-RoPE, $G = 8$	R@3 0.250	Token-QK order correlation 0.874	Coordinate repair does not create evidence semantics
Question decay, strongest tested	R@3/MRR 0.250/0.279	Last state 0.469/0.413	More query tokens are not automatically a better query
Width 256 and harder negatives	No validation-selected gain	Twice the 128-wide index at width 256	Data and objective generalization dominate raw capacity

The chunk-size sweep supplies a related caution. At 128-token chunks and $k = 8$, recall reaches 0.688 but the selected fraction is 41.3%; at $k = 16$, 71.5% of parents are selected. Coarser chunks can improve apparent fixed-k recall by making each choice cover more source text. That is not a sparse win, which is why the main paper reports selected fractions and physical K/V.

Independent hidden-state confirmation over 64 evaluations gives any/all-evidence recall of 0.391/0.063 at $k = 3$, 0.578/0.203 at $k = 8$, and 0.797/0.500 at $k = 16$. The selected-parent fractions are 0.045, 0.119, and 0.238, while the hard materialization budget holds the latter two near 0.059 physically active K/V. This illustrates all three distinctions at once: any versus all evidence, selection versus materialization, and fixed rank versus selected fraction.

B Code-Level Adapter Reimplementation Guide

This appendix gives the minimum code structure needed to reproduce the integration without copying the Qwen implementation. Source references use physical line numbers at repository commit `db50846`. This checkpoint makes attention-input hidden-state routing the explicit default and exposes the fixed-selection materialization boundary. The line references are deliberately revision-pinned because upstream Transformers APIs and local diagnostics will move over time. The multi-layer identity mapping, experiments, tests, and complete artifacts are pinned separately at commit `8f6ef41`; all multi-layer line references below refer to that revision.

B.1 Tensor and ownership contract

A new family should add projection, position, cache, mask, kernel, and output conventions in its adapter, while leaving selection and memory policy in the shared core.

The canonical tensor contract is

$$Q \in \mathbb{R}^{B \times H_q \times T_q \times d_h}, \qquad K, V \in \mathbb{R}^{B \times H_{kv} \times T_k \times d_h}. \tag{8}$$

Reference entries use $B = 1$ and keep H_{kv}, including for GQA or MQA. They are not expanded to H_q in persistent storage. The controlled replay implementation shows the only required expansion point in `memory_batching.py`, lines 216–290: after memory and local K/V are joined, native K/V heads may be repeated for the attention computation. Qwen's installed eager kernel performs the corresponding operation in the production adapter.

B.2 Minimal forward-path skeleton

The following pseudocode is the adapter's essential control flow. It is intentionally family-neutral; "native" operations must call the installed model implementation rather than reimplementing its mathematics.

```
01  forward(hidden, native_positions, native_mask, past):
02      if capture_is_armed:
03          q_pre, k_pre, v = native_project_and_norm(hidden)
04          q_post, k_post = native_position(q_pre, k_pre)
05          save_transient_routing_features(hidden, q_pre, q_post, k_pre)
06          save_post_position_detail_kv(k_post, v)
07
08      if memory_is_disabled or reference_cache_is_empty:
09          return original_attention(hidden, native_positions,
10                                    native_mask, past)
11
12      q_pre, k_pre, v = native_project_and_norm(hidden)
13      q_post, k_post = native_position(q_pre, k_pre)
14      if past is not None:
15          k_post, v = past.update(k_post, v, layer_id, native_kwargs)
16      route_q = matched_query(hidden[:, -1, :], q_pre, q_post)
17      memory = shared_core.prepare_memory(q_post, route_q,
18                                          direct_tokens=k_post.shape[2])
19      if memory.is_empty:
20          return native_attention_and_output(q_post, k_post, v, native_mask)
21      k, v, mask = shared_core.combine_local_and_memory_kv(
22          k_post, v, memory, native_mask, query_tokens=q_post.shape[2])
23      return native_attention_and_output(q_post, k, v, mask)
```

Lines 8–10 are a correctness boundary, not an optimization. The Qwen implementation delegates to the untouched module at `hf/qwen.py`, lines 190–201, which makes disabled behavior auditable and bit-exact. In the active path, projection and RoPE occur at lines 204–207 and `Cache.update` occurs once at lines 208–215. Lines 216–221 choose the matched routing query without altering the post-RoPE attention query. If routing returns no materialized memory, lines 224–234 call the native kernel directly. Delegating at that point would execute a second cache update and duplicate generation history.

Memory is composed only after the native cache update. The shared operation at `core.py`, lines 394–436, pads variable memory rows, prepends memory K/V, creates an all-visible additive

mask for valid memory positions, masks padding, and concatenates the unchanged local mask. The adapter then invokes Qwen's eager function and one pretrained `o_proj` at `hf/qwen.py`, lines 92–110 and 246–258. Thus local and memory positions share one softmax and one output projection.

B.3 Optional learned routing metric

The learned adapter is a replaceable routing component, not part of native attention. For an asymmetric linear metric,

$$r_q = \mathrm{norm}(W_q h_q), \qquad r_c = \mathrm{norm}(W_c g_c), \qquad s(q,c) = r_q^\top r_c. \tag{9}$$

A shared adapter aliases $W_q = W_c$. An independent implementation needs only the following boundary, revision-pinned at commit `a657bc7`:

```
# Reference publication, after ordinary hidden-state gist pooling
routing_gist = adapter.project_memory(routing_gist)
cache.store(routing_gist, native_post_rope_k, native_v)

# Live routing; native attention continues to use q_post
information_need = aggregate_query_states(h_in, strategy)
routing_query = adapter.project_query(information_need)
selected_ids = cache.search(routing_query)
k_mem, v_mem = cache.materialize_native_kv(selected_ids)
native_attention(q_post, concat(k_mem, k_local),
                  concat(v_mem, v_local))
```

The projection implementation is `hf/routing_adapter.py`, lines 10–74; checkpoint loading and freezing are lines 77–94. Memory projection occurs at `hf/injection.py`, lines 238–246, and live query projection at `hf/qwen.py`, lines 188–202. Cast hidden features to the adapter parameter dtype at this boundary; the checkpoint is FP32 while Qwen emits FP16 states. Do not cast, project, or recompute native detail K/V. Their exact-equality invariant is the test that prevents a routing retrofit from silently becoming an attention rewrite.

B.4 Oracle selection and causal diagnostics

An oracle must replace only identity selection. Adding a separate K/V injection path would confound semantic correctness with different budgeting, positions, masks, or attention code. The operational handle activates a layer set and assigns optional row-local identities in `hf/injection.py`, lines 69–89. The Qwen adapter chooses ordinary routing or the fixed set at `hf/qwen.py`, lines 273–286; both enter `core.py::prepare_selected_memory`, lines 337–388. The latter still applies the trigger, deduplication, whole-chunk budget, overlap policy, residency transfer, and rectangular packing. The fixed-set setter and its ownership contract are in `hf/adapter_base.py`, lines 71–83.

The experiment constructs oracle identities by intersecting annotation token spans with parent logical spans in `run_oracle_memory_use.py`, lines 107–134. Lines 194–281 append the gold continuation, gather its autoregressive token probabilities, retain opt-in eager-attention weights, and use temporary decoder-layer hooks for answer-position hidden states. Attention capture is disabled immediately afterward and is never part of ordinary inference. Conditions and paired deltas are assembled at lines 310–419. A reimplementation should assert three invariants per example: requested oracle IDs equal retained IDs, budget rejection is zero, and the largest native operation stays below the declared bound.

B.5 Route-once multi-layer consumption

Multi-layer PRA shares an identity decision, not an attention tensor. An implementation should make the distinction visible in its types:

```
selected = route_once(query_at_layer_27)       # parent IDs + scores
for layer in consumption_layers:
    layer_hits = []
    for hit in selected:
        chunk = cache[hit.uri].layer[layer].by_id[hit.chunk_id]
        assert chunk.kv.was_projected_by == layer
        layer_hits.append(hit.with_payload(chunk.kv))
    adapter[layer].set_fixed_selection(layer_hits)
model(question)                                # no further semantic search
```

The operational API activates an arbitrary layer set at `hf/injection.py`, lines 70–93. Identity-to-payload resolution is lines 97–145: for each requested layer it looks up the stable `chunk_id` in that layer's `LayerReferenceMemory`, replaces the payload, and records the source routing layer. A missing layer or chunk is an error rather than a fallback to another layer's K/V. The Qwen adapter's fixed-selection branch at `hf/qwen.py`, lines 273–289 then calls the same budgeting, transfer, mask, and attention path as ordinary routing.

The experiment captures the last-layer information-need state and performs one complete ranking at `run_multilayer_pra.py`, lines 161–198. Oracle and routed identities are mapped at lines 277–325; the remainder of the function records per-layer attention, hidden-state changes, materialized tokens, transfers, and latency. The invariant test at `tests/test_hf_integration.py`, lines 548–586 asserts equal parent IDs but different target-layer objects, tensor pointers, and values while preserving native $[B, H_{kv}, T, d_h]$ shape. This catches the most dangerous superficially working implementation: copying $K^{(27)}, V^{(27)}$ into every consuming layer.

B.6 Publishing a reference

Reference construction is a bounded prefill, not a second attention mechanism. A portable implementation follows this recipe:

```
disable_PRA_memory_during_reference_encoding()
for [block_start:block_end] in bounded_encoding_blocks(source):
    position_ids = logical_source_positions(block_start, block_end)
    arm_post_position_kv_capture(position_ids)
    model(block_ids, position_ids=position_ids, use_cache=False)
    captured = consume_native_kv_for_each_PRA_layer()
    for routing_window in overlapping_windows(captured):
        gist = pool(routing_window.attention_input_hidden_state)
        publish(uri, logical_offsets, post_rope_k, native_v, gist)
        discard_token_level_hidden_state()
restore_previous_memory_state()
```

The concrete loop is `hf/injection.py`, lines 235–354. Encoding block size limits one native model invocation; routing chunk size and overlap independently determine index granularity (lines 266–292). Each chunk slices the captured post-RoPE K/V, chooses transient routing features

through `_routing_points` (lines 172–233), computes one or more compact gists, and records logical source offsets (lines 293–348). Keeping those two scales separate is important: changing index granularity must not silently change how the pretrained model encoded the source.

The contiguous multi-gist extension keeps that ownership boundary explicit. For a requested count G, split the routing feature rows into balanced source-order ranges, mean each range, score all resulting $[G, D]$ rows, reduce to one parent score, and preserve the winning local index for diagnostics. Top-k must then operate on parent identities before budgeting; otherwise several winning gists could duplicate one parent K/V payload. The reference implementation is revision-pinned at `0205df8`: pooling is in `gists/segment_mean.py`, lines 11–56, publication remains `hf/injection.py`, lines 163–190, and packed scoring plus max reduction remains `memory.py`, lines 487–593.

The same operation implements displaced long-prompt history. At `hf/injection.py`, lines 357–370, the prefix is published under the reserved `#__head` URI and the direct tail retains continued logical position IDs. Every reference-encoding call and every direct call must satisfy the model-safe native bound even when the logical source is longer.

B.7 Porting another decoder family

A practical port begins by locating seven hooks in the installed family: the attention module inside each decoder layer; Q/K/V projections; any Q/K normalization; positional encoding; generation-cache update and its keyword arguments; native mask convention; and the eager attention plus output projection. Implement the six abstract methods declared at `hf/adapter_base.py`, lines 85–106, then add a narrow type-selection branch analogous to `inject_pra` at `hf/injection.py`, lines 379–428. No checkpoint conversion or parameter copy should be needed.

The recommended validation order is:

1. With memory disabled, require exact logits, hidden states, greedy tokens, and generation- cache lengths against an unwrapped copy (test lines 55–72).

2. Capture a short reference and verify post-position state, logical offsets, device residency, and H_{kv} rather than H_q storage (lines 74–93).

3. Verify matched pre/post capture, representation switching, unchanged post-RoPE detail K/V, GQA grouping, serialization, and device parity (lines 109–214).

4. Supply one selected identity through router and oracle traces; require identical native K/V and attention output (lines 241–280).

5. Compare GQA replay with explicit head repetition on a synthetic tensor (lines 216–239).

6. Exercise an over-limit logical prompt and verify bounded native operations plus continued position IDs (lines 282–313).

7. Generate with active memory and assert that direct cache length grows by exactly one per decoded token (lines 315–330).

Four mistakes deserve explicit tests. Applying position encoding again to stored post-position keys changes their basis. Permanently expanding GQA K/V multiplies memory without adding information. Replacing the family mask with a generic causal mask can alter padding or sliding-window behavior. Finally, calling the wrapped module after manually updating its cache appends the same local token twice. Passing transport and parity tests establishes a faithful adapter; it does

not establish that the router selects causally useful evidence, which requires the separate quality controls reported in the main paper.

B.8 Llama port and product API

The release implementation is pinned at commit `809694b`. The common abstract family contract remains in `adapter_base.py`, lines 32–132. Qwen's implementation resolves its installed helper functions through an overridable method at `qwen.py`, lines 38–77. The Llama module at `llama.py`, lines 8–48, subclasses that shared projection/capture/control path, replaces the symbol resolver, and invokes Llama's native eager function without the Qwen sliding-window argument. This is inheritance of a family-neutral tensor contract, not reuse of Qwen parameters. Injection selects Qwen or Llama from the concrete attention module at `injection.py`, lines 380–430.

An independent family port should make the same decision only after comparing installed forward signatures and testing every shared assumption. For Llama those assumptions are: Q/K/V linear outputs reshape to $[B, H, T, d_h]$; RoPE receives the same cosine/sine tuple; cache update accepts the layer index, sine, cosine, and cache position; the eager kernel repeats native K/V heads internally; and output returns token-major states before one native output projection. If any family differs, override that specific abstract method rather than adding a family branch to the router, cache, or budgeter. Tests in `test_hf_llama_integration.py`, lines 51–106, cover exact disabled behavior, native-head reference payloads, active memory, and rollover.

The public layer composes rather than replaces those internals. `pra_hf/config.py`, lines 14–127, validates stable product names, resolves negative layer indices, and translates them to the core configuration. `pra_hf/router.py`, lines 14–110, defines the versioned weights/metadata/model-card artifact and legacy-checkpoint conversion. The high-level model in `pra_hf/model.py`, lines 42–358, owns tokenizer/model loading, router compatibility, reference lifecycle, fraction selection, route-once identity replay, generation, chat, and memory economics. In particular, lines 192–258 flatten complete reference-grouped rankings, apply a global fraction or top-k cutoff, reconstruct typed identities, and map those IDs to each consuming layer's native payload. The CLI in `pra_hf/cli.py`, lines 22–145, is a thin caller of the same API; it does not maintain a second execution path. Product tests at `test_pra_hf_api.py`, lines 66–132, pin config serialization and precedence, router save/load and freezing, add/remove/clear, fraction routing, generation, statistics, and the rule that a router cannot change after gists have been indexed.

B.9 Revision-pinned source map

Table 16 connects the preceding control flow to the implementation.

Table 16: Source map for an independent adapter implementation.

Responsibility	File and physical lines
Abstract family boundary	`src/pra_torch/hf/adapter_base.py`, 16–106
Qwen projections, RoPE, native kernel, and output	`src/pra_torch/hf/qwen.py`, 53–110
Llama native-symbol binding and eager invocation	`src/pra_torch/hf/llama.py`, 8–48
Matched routing/detail capture	`src/pra_torch/hf/qwen.py`, 109–148; `src/pra_torch/hf/injection.py`, 80–210
Disabled and active runtime paths	`src/pra_torch/hf/qwen.py`, 171–275
Routing query and GQA reduction	`src/pra_torch/core.py`, 92–121
Budget, deduplication, transfer, and materialization	`src/pra_torch/core.py`, 155–392
Fixed-selection identity/payload boundary	`src/pra_torch/core.py`, 335–392
Route-once layer-native identity mapping	`src/pra_torch/hf/injection.py`, 70–145
K/V and additive-mask composition	`src/pra_torch/core.py`, 394–436
Reference publication and selected-layer injection	`src/pra_torch/hf/injection.py`, 235–428
Depth schedules and causal protocol	`experiments/paper2_hf/qa/run_multilayer_pra.py`, 62–964
Parity, routing, GQA, head, cache, and selection gates	`tests/test_hf_integration.py`, 80–636
Stable config, router artifact, and public API	`src/pra_hf/config.py`, 14–127; `src/pra_hf/router.py`, 14–110; `src/pra_hf/model.py`, 42–358
Llama and product-facing gates	`tests/test_hf_llama_integration.py`, 51–106; `tests/test_pra_hf_api.py`, 66–132

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[2] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: `https://arxiv.org/abs/2105.03011`.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[5] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[6] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop

question answering. In *Proceedings of EMNLP*, 2018. Preprint: https://arxiv.org/abs/1809.09600.

[7] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

[8] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: https://openreview.net/forum?id=RkRrPp7GKO.